Maxwell Alisson Soares Doroteu

Projeto de automação

🛒 **Automação de Testes E2E - Magento https://magento2-demo.magebit.com/ - Luma**

Status: ✅ Completo
Tipo: Automação E2E
Framework: Robot Framework

📖 Sobre o Projeto

Este projeto contém a suíte de testes automatizados de ponta a ponta (End-to-End) para os fluxos críticos da plataforma Magento 2. O objetivo principal é garantir a qualidade e a integridade das jornadas de Cadastro, Autenticação, Carrinho de Compras e Checkout (Finalização de Pedido).
A arquitetura de testes foi desenvolvida para simular a interação real do usuário, lidando inclusive com preenchimento de campos dinâmicos de endereço e formulários complexos na interface do sistema.

📂 Estrutura do Projeto

```
.
├── Funcionalidades/
│   ├── Login.resource
│
├── Metodos_Globais/
│   ├── Metodos_Globais.resource
│
├── Pages/
│   ├── Componentes/
│ │   ├── Login.resource
│ │   ├── Menu_Lateral.resource
│ │
│   ├── Login_page.resource
│
├── Resources/
│   ├── Base.resource
│   ├── ScreenshotLibrary.py
│
├── TestData/
│   ├── JSON_config.resource
│
├── Testes/
│   ├── Login.robot
│
├── results/
│
├── venv/
│
├── .gitignore
├── robot.toml
├── interactive_console_output.xml
```

📌 Organização das Pastas

- Funcionalidades: Contém as implementações das features de negócio (ex: login)
- Metodos_Globais: Métodos reutilizáveis e genéricos utilizados em diferentes fluxos
- Pages: Representação das páginas e componentes (Page Object Model)
- Resources: Configurações base, setup e bibliotecas customizadas
- TestData: Massa de dados e configurações externas
- Testes: Casos de teste automatizados
- results: Relatórios gerados após execução e Screenshots de evidência

🧩 Boas Práticas Aplicadas

- Reutilização de keywords para evitar duplicação de código
- Separação entre lógica de teste e dados
- Uso de tags para execução seletiva de cenários
- Estrutura modular visando manutenção e escalabilidade

📊 Relatórios

Após a execução, são gerados automaticamente:

- log.html → detalhamento completo dos passos
- report.html → resumo da execução

🛠️ Stack Tecnológico

O framework de automação foi construído utilizando as seguintes ferramentas e bibliotecas:

- **Linguagem:** Python 3.x
- **Framework Core:** Robot Framework
- **Interação Web:** SeleniumLibrary
- **Manipulação de Dados:** String Library
- **Massa de Dados Dinâmica:** FakerLibrary (utilizada para geração de e-mails e dados de usuários únicos a cada execução)
- **Interação de Interface (Desktop/OS):** pyautogui (utilizado para mapear e interagir com elementos dinâmicos de formulário, como inserção de quadras, conjuntos e números residenciais em campos de endereço).
  O uso do PyAutoGUI foi adotado como workaround para limitações específicas de interação com campos dinâmicos não acessíveis via DOM, porém em cenários reais recomenda-se priorizar soluções baseadas em Selenium puro ou APIs.

⚙️ Pré-requisitos e Configuração do Ambiente

Antes de executar os testes, certifique-se de ter as seguintes dependências instaladas na sua máquina:

1. **Python 3 instalado e configurado nas variáveis de ambiente (PATH).**
2. **Navegador Web** (Google Chrome ou Mozilla Firefox atualizados).
3. **Webdriver** correspondente ao navegador (ex: chromedriver) configurado no PATH do sistema.
4. **Instalação das Bibliotecas via PIP:** Abra o terminal e execute o comando abaixo para instalar todas as dependências do projeto:

Bash

```
pip install robotframework robotframework-seleniumlibrary robotframework-faker pyautogui
```

Atenção (Uso do PyAutoGUI): Durante a execução do CTA_06, que utiliza comandos de automação gráfica (mouse e teclado via OS), a tela do computador **não deve ser bloqueada** ou minimizada, e o foco deve ser mantido no navegador para garantir que o preenchimento dos dados ocorra perfeitamente.

🚀 Possível Evolução

- Integração com CI/CD (GitHub Actions, Jenkins)
- Execução headless
- Execução paralela

▶️ Como Executar os Testes

Para rodar a suíte completa de testes, abra o terminal no diretório raiz do projeto e execute:

Bash

```
robot -d ./logs nome_do_arquivo.robot
```

- O parâmetro -d ./logs garantirá que os relatórios em HTML (log.html e report.html) gerados pelo Robot Framework sejam salvos em uma pasta organizada.

Para executar um cenário de teste específico (por exemplo, apenas o fluxo de adição ao carrinho), utilize o comando usando a tag ou o nome do teste:

Bash

```
robot -d ./logs -t "CTA_05_Adicionar_Produto_Carrinho" nome_do_arquivo.robot
```

🚀 Fluxo de Execução Inteligente (Massa de Dados)

Um dos grandes diferenciais desta automação é a **independência de intervenção manual**. O projeto foi estruturado para que o usuário precise apenas "dar o play" na execução.

- **Autossuficiência de Dados:** A suíte utiliza o **CTA_01 (Cadastro de Usuário)** como o estágio inicial obrigatório. Este cenário utiliza a FakerLibrary para criar um perfil de usuário único e aleatório a cada rodada.

- **Encadeamento Dinâmico:** Os dados gerados (e-mail e senha) no primeiro teste são capturados e repassados automaticamente para os scripts subsequentes (**CTA_02** ao **CTA_06**).

- **Intervenção Zero:** Como a massa de dados é volátil e criada em tempo real, não há necessidade de preparar planilhas de usuários ou realizar cadastros manuais prévios no ambiente. O sistema garante que os testes de Login e Checkout sempre utilizem uma conta válida e recém-criada, evitando falhas por dados expirados ou duplicados.

📋 Requisito de Execução Automática

Para que a automação flua de ponta a ponta sem erros, os testes devem ser executados na ordem numérica (ou a suíte completa), garantindo que o ciclo de vida do dado (Criação -> Login -> Compra) seja respeitado de forma 100% automatizada.

📝 Cenários de Teste Automatizados (BDD)

Abaixo estão descritos os cenários validados nesta suíte, escritos no formato Behavior-Driven Development (BDD) para facilitar a compreensão por todos os envolvidos no ciclo de desenvolvimento.

🥷 Épico: Gestão de Contas (Cadastro e Login)

CTA_01_Cadastro_Usuario

- **Documentação:** Este script de teste tem por objetivo validar o cadastro de novo usuário com geração de massa de dados dinâmica.
- **Passos:**
    - **Dado** que eu acesse a página inicial da loja
    - **Quando** aciono o botão "Create an Account"
    - **E** preencho o campo "First Name"
    - **E** preencho o campo "Last Name"
    - **E** preencho o campo "Email" com email aleatorio
    - **E** preencho o campo "Password" e "Confirm Password"
    - **E** aciono o botão "Create an Account" para registro da conta
    - **Então** a conta de usuário é criada

CTA_02_Login

- **Documentação:** Este script de teste tem por objetivo validar o login com credenciais válidas ao sistema de compras online, usando os dados dinâmicos que são no CTA_01.
- **Passos:**
    - **Dado** que eu acesse a página inicial da loja
    - **Quando** aciono o botão "Sign In"
    - **E** preencho o campo "Email" e "Password" para logar
    - **E** aciono o botão "Sign In" para login
    - **Então** o usuário é logado e redirecionado para a página inicial do site

CTA_03_Login_Senha_Invalida_Fluxo_Excecao

- **Documentação:** Este script de teste tem por objetivo validar o comportamento do sistema ao informar uma senha incorreta durante o login.
- **Passos:**
    - **Dado** que eu acesse a página inicial da loja
    - **Quando** aciono o botão "Sign In"
    - **E** preencho o campo "Email" e "Password" para login
    - **E** aciono o botão "Sign In" para login
    - **Então** é apresentada a mensagem *"The account sign-in was incorrect or your account is disabled temporarily. Please wait and try again later"*

CTA_04_Login_Email_Invalido_Fluxo_Excecao

- **Documentação:** Este script de teste tem por objetivo validar o comportamento do sistema ao informar um e-mail incorreto/inexistente durante o login.
- **Passos:**
    - **Dado** que eu acesse a página inicial da loja
    - **Quando** aciono o botão "Sign In"
    - **E** preencho campo "Email" e "Password" para login
    - **E** aciono o botão "Sign In" para login
    - **Então** é apresentada a mensagem *"The account sign-in was incorrect or your account is disabled temporarily. Please wait and try again later"*

🛍️ Épico: Jornada de Compra (Carrinho e Checkout)

CTA_05_Adicionar_Produto_Carrinho

- **Documentação:** Este script de teste tem por objetivo validar a adição de um produto válido ao carrinho de compra logado, usando os dados dinâmicos que são criados no CTA_01.
  - **Passos:**
    - **Dado** que eu acesse a página inicial da loja
    - **Quando** aciono o botão "Sign In"
    - **E** preencho o campo "Email" e "Password" para logar
    - **E** aciono o botão "Sign In" para login
    - **E** seleciono um item para compra na pagina inicial
    - **E** adicionar ao carrinho
    - **Então** o produto é adicionado ao carrinho

CTA_06_Finalizacao_Compra_Checkout

- **Documentação:** Este script de teste tem por objetivo validar o fluxo completo (End-to-End) de checkout e finalização de pedido de um produto previamente adicionado, usando os dados dinâmicos que são criados no CTA_01.
- **Passos:**
  - **Dado** que eu acesse a página inicial da loja
  - **Quando** aciono o botão "Sign In"
  - **E** preencho o campo "Email" e "Password" para logar
  - **E** aciono o botão "Sign In" para login
  - **E** seleciono um item para compra na pagina inicial
  - **E** visualizo o carrinho com o produto já adicionado do step anterior
  - **E** aciono o botão "Proceed to Checkout"
  - **E** preencho os campos obrigatórios para "Endereço de envio"
  - **E** aciono o botão "Next"
  - **E** aciono o botão "Place Order" para finalização da compra
  - **Então** a compra é finalizada e é apresentado a mensagem de sucesso na tela *"Thank you for your purchase!"*